

中山大学计算机学院

人工智能

本科生实验报告

课程名称: Artificial Intelligence

学号	22336203	姓名	沈苇杭
----	----------	----	-----

一、实验题目

实验 2 知识表示与推理

二、实验内容

1. 算法原理

单步归结: 从两个待归结的子句中选择原子及其否定, 去掉这个原子并将两个子句合并为一个。

最一般合一: 合一算法是指通过变量替换, 使得两个子句拥有相同的原子, 可以归结。最一般合一是最简单的一组变量替换。最一般合一的操作方法是: 找到两个原子公式的第一个不匹配项, 倘若这两项一个是变量, 一个是常量, 就把变量赋值为常量, 并将两个子句中相同的变量都赋值为这个常量, 并更新赋值集合。

归结算法原理: 将待归结的子句集转化为方便计算机处理的 casual form (在题目中已经转换), 重复最一般合一+单步归结的步骤, 直至归结出空子句或归结不出新子句。

2. 关键代码展示

1.

```

25 # 子句数目和每个子句中的谓词数目
26 num_clause = int(input())
27 num_words = []
28 for i in range(num_clause):
29     num_words.append(int(input()))
30 #print(num_words)
31
32 # 输入KB,分割KB, 存为元组和集合, 另外存一个列表方便操作
33 print("请输入字句集, 注意不要输入 'KB=' 和大括号, 并且所有的输入需要在一行内完成")
34 s = input()
35 origin = (re.findall( pattern: r'~?\w+\.(\w+\.)*\w*\.', s))
36 #print(origin)
37 clause_set = set()
38 clause_list = list()
39 b = 0
40 a = 0
41 while b < sum(num_words):
42     temp_list = origin[b:b+num_words[a]]
43     temp_tuple = tuple(temp_list)
44     print(temp_tuple)
45     clause_set.add(temp_tuple)
46     clause_list.append(temp_list)
47     b = b + num_words[a]
48     a = a+1
49 #print(clause_list)
50 ResolutionFOL(clause_list)
51

```

输入与输出：借助 re.findall 函数和正则表达式分割输入的字符串，并将它们转化为元组，添加进字典。

2.

```

def unifier(l1, l2):
    v_list = ['xx', 'yy', 'x', 'y', 'z', 'w', 'u', 'v']
    s = '0'
    for t in range(len(l1)):
        #不能合一, 返回0
        if v_num(l1) > 1 or v_num(l2) > 1:
            return s
        if t > 0 and l1[t] not in v_list and l2[t] not in v_list and l1[t] != l2[t]:
            return s
        #需要合一, 且只有一个变量
        if l1[t] != l2[t] and (l1[t] in v_list or l2[t] in v_list):
            if l1[t] in v_list:
                if l2[t] in v_list:
                    l1[t] = l2[t]
                    s = l2[t]
                    return s
                else:
                    l1[t] = l2[t]
                    s = l2[t]
                    return s
            else:
                if l2[t] in v_list:
                    l2[t] = l1[t]
                    s = l1[t]
                    return s
        #除了符号都相同, 返回1
        if (l1[0] == '~' + l2[0] or l2[0] == '~' + l1[0]) and equals(l1, l2) == 1:
            s = '1'
    return s

```

```

80 def unify(list5, target):
81     v_list = ['xx', 'yy', 'x', 'y', 'z', 'w', 'u', 'v']
82     list6 = list5.copy()
83     for d in range(len(list5)):
84         #转化为方便处理的形式
85         if list5[d] != ' ':
86             temp = trans(list5[d])
87             for f in range(len(temp)):
88                 if temp[f] in v_list:
89                     #将变量变成目标量(变量或常量)
90                     temp[f] = target
91             list6[d] = anti_trans(temp)
92     return list6
93

```

最一般合一：最一般合一由两个函数完成。

第一个函数的输入是两个具有相同谓词、不同符号的原子。若两个原子都不含

变量且常量不相同，函数返回字符‘0’；若两个原子的变量相同，函数返回字符‘1’；若两个原子一个含变量，一个含常量，或含有不同的变量，则进行赋值，函数返回赋的常量。

第二个函数的输入是一个子句和第一个函数里赋给原子的值。函数查找相同的变量，将第一个函数返回的常量赋给这个变量。

3. 归结

```
while l < len(clause_list1[k]):
    # 选出要比较的
    tmp_c1 = clause_list1[i][j]
    tmp_c2 = clause_list1[k][l]
    if all_space(tmp_c1) == 0 and all_space(tmp_c2) == 0:
        a = same(tmp_c1, tmp_c2)
        if a == 1 and tmp_c1[0] != tmp_c2[0]:
            list1 = trans(tmp_c1)
            list2 = trans(tmp_c2)
            # unify, 归结, 输出
            y = unifier(list1, list2)
            if y == '1':
                #除了符号都相同, 直接归结
                tlist1 = clause_list1[i].copy()
                tlist2 = clause_list1[k].copy()
                tlist1[j] = ' '
                tlist2[l] = ' '
                tlist1.extend(tlist2)
                tlist1 = differ(tlist1)
                clause_list1.append(tlist1)
                show(i, j, k, l, tlist1)
                if all_space(tlist1) == 1:
                    print("结束")
                    return
            if y != '1' and y != '0' and list1[0] != list2[0]:
                #赋值
                tlist1 = unify(clause_list1[i], y)
                tlist2 = unify(clause_list1[k], y)
                tlist1[j] = ' '
                tlist2[l] = ' '
```

以上代码展示了归结的关键步骤。找到要比较的两个原子，如果这两个原子的谓词相同，则对两个原子调用第一个合一函数。倘若第一个函数返回‘1’，那么直接归结，将新子句添加进子句集。倘若第一个函数返回赋的值，那么对两个子句调用第二个合一函数，再归结。

4. 辅助函数

`def trans(string):` : 将原子转换为方便处理的列表。

`def anti_trans(list7):` : 将列表转换回原来的形式。

```
def equals(l1,l2):  
    #print(l1,l2)    : 判断两个原子的参数项是否完全相同。
```

```
def same(string1, string2):|  
    : 判断两个原子的谓词是否相同。
```

```
def v_num(list8):  
    : 计算原子的参数数目。
```

```
def differ(list1):  
    : 去除子句中相同的原子。
```

```
def all_space(list3):  
    : 判断子句是否为空。
```

```
def show(clause_list1,i,j,k,l,tlist3,y,v1,v2):  
    : 打印
```

推理步骤和结果。

三、 实验结果及分析

1. 实验结果展示示例

```
E:\学习\Pycharm\AILab2\venv\Scripts\python.exe E:\学习\Pycharm\AILab2\main.py  
4  
1  
2  
2  
1  
请输入子句集, 注意不要输入'KB='和大括号, 并且所有的输入需要在一行内完成  
(GradStudent(sue),), (~GradStudent(x), Student(x)), (~Student(x), HardWorker(x)), (~HardWorker(sue),)  
(GradStudent(sue)',)  
(~GradStudent(x)', 'Student(x)')  
(~Student(x)', 'HardWorker(x)')  
(~HardWorker(sue)',)  
R[0,1a]={0=sue}('Student(sue)',)  
R[1a,0]={x=sue}('Student(sue)',)  
R[1b,2a]={x=1}('~GradStudent(x)', 'HardWorker(x)')  
R[2a,1b]={x=1}('~GradStudent(x)', 'HardWorker(x)')  
R[2a,4]={x=sue}('HardWorker(sue)',)  
R[2a,5]={x=sue}('HardWorker(sue)',)  
R[2b,3]={x=sue}('Student(sue)',)  
R[3,2b]={0=sue}('~Student(sue)',)  
R[3,6b]={0=sue}('~GradStudent(sue)',)  
R[3,7b]={0=sue}('~GradStudent(sue)',)  
R[3,8]=()  
结束  
  
Process finished with exit code 0
```

输入子句集长度, 每个子句长度, 子句集内容, 输出每个子句、推理过程。

2. 评测指标展示及分析

此实验中没有评测指标。

四、 参考资料

[正则表达式中的*, +, ?, ^, \\$, 范围和次数用法_正则表达式-CSDN 博客](#)